



دانشکده علوم ریاضی

رشته علوم کامپیوتر

گزارش تمرین 2

محاسبه مساحت با روش مونت-کارلو

از طریق MPI

تهیه کننده:

ماریا صبوری دودران

شماره دانشجویی: 402222001

خرداد ماه

فهرست

3	توضیح کد
6	توضیح نتایج
6	برسی دقت محاسبه
8	برسی speedup و efficiency

توضیح کد

برای اینکه حجم کد در تابع main() کاهش یابد، ابتدا مقدار ورودی را توسط process 0 در تابع get_input() دریافت کردم و از طریق MPI_BCAST به بقیه process ها مخابره کردم:

```
void get_input(
    int rank_num,
    int comm_sz,
    double* a_p,
    double* b_p,
    int* n_p,
    int* mont_carlo_n,
    double* h,
    time_t* start_time
){
    if (rank_num == 0){
        printf("Enter a, b, n and mont_carlo_n\n");
        scanf("%lf %lf %d %d", a_p, b_p, n_p, mont_carlo_n);
    }
    *h = (*b_p - *a_p) / *n_p;

    MPI_Bcast(a_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Bcast(b_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Bcast(h, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Bcast(n_p, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(mont_carlo_n, 1, MPI_INT, 0, MPI_COMM_WORLD);

    if (rank_num == 0) {
        time(start_time);
    }
}
```

با گرفتن n، مشخص میشود که میخواهیم چه تعداد زیرمساحت داشته باشیم. این تعداد زیرمساحت ها باید در process ها توزیع شود. نحوه ی این توضیح همانند روشی است که برای scatterv انجام میدهیم. این محاسبات در تابعی به نام distribute_local_values انجام شده است. نحوه ی توزیع زیرمساحت ها به گونه ای است که هر دو process حداکثر به تعداد یک زیر مساحت نیاز به محاسبه بیشتر داشته باشد.

```
void distribute_local_values(
    int rank_num,
    int comm_size,
    double a,
    double b,
```

```

double h,
double* local_a,
double* local_b,
int n,
int* local_n
){
    /*calculate local_n*/
    *local_n = n / comm_size;
    int remainder = n % comm_size;
    *local_n += (rank_num < remainder) ? 1 : 0;

    /*calculate local_a and local_b*/
    *local_a = a + (rank_num * (*local_n) + ((rank_num >= remainder) ? remainder:0)) * h;
    *local_b = *local_a + (*local_n) * h;

    printf("rank: %d ,local a: %lf, local b: %lf, h: %lf, local n: %d\n",rank_num, *local_a, *local_b, h, *local_n);
}

```

زمانی که هر **process** متوجه شد که در چه بازه ای باید محاسبات را انجام دهد و به چه تعداد، تابع **monte_carlo** را اجرا میکنند. منطق این تابع به این صورت است که برای هر زیرمساحت، 2 عدد رندم نرمال گرفته میشود. سپس این 2 عدد نرمال را در اسکیل عمودی (بین 0 تا حداکثر ارتفاع آن زیر مساحت¹) و اسکیل افقی (بین 2 بازه ی زیرمساحت) میبریم. اگر این مختصات داخل تابع افتاده باشد، به مقدار متغیر **hit** یک واحد اضافه میشود. در صورتی که به تعداد مشخص شده (**mont_carlo_n**) آزمایش انجام شد، مساحت زیرمساحت توسط رابطه زیر محاسبه میشود:

$$\frac{\int_a^b f(x)dx}{(b-a)F_{MAX}} = \frac{hit}{mont}$$

بعد از اینکه **process** مساحت های زیرمساحت های خود را محاسبه کرد، توسط **MPI_REDUCE** نتایج به **process0** ارسال میشود تا جمع نتایج همه **process** ها را انجام دهد.

```

void mont_carlo(
    int rank_num,
    int comm_sz,
    double left_endpt,
    double right_endpt,
    double h,
    int local_n,
    int mont_carlo_n,
    double* total_area,

```

¹ در تابع **max_func** این حداکثر ارتفاع زیر مساحت محاسبه میشود.

```

double* local_area
){
    srand(time(NULL) + rank_num);
    // printf("rank: %d\n", rank_num);
    *local_area = 0;
    for (int i = 0; i < local_n; i++) {
        int hit = 0;

        for (int j = 0; j < mont_carlo_n; j++) {
            double r_x_normalized = (double) rand() / RAND_MAX;
            double r_y_normalized = (double) rand() / RAND_MAX;

            double r_x = left_endpt + i * h + h * r_x_normalized;
            double r_y = max_func(left_endpt + i * h, left_endpt + (i + 1) * h )
* r_y_normalized;

            hit += (U(r_x) >= r_y) ? 1 : 0;
        }
        *local_area += max_func(left_endpt + i * h, left_endpt + (i + 1) * h ) *
h * ((double)hit / mont_carlo_n);
    }
    printf("rank: %d, left_endpt: %lf, right_endpt: %lf, local area: %lf\n",
rank_num, left_endpt, right_endpt, *local_area);
    MPI_Reduce(local_area, total_area, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);
}

```

توضیح نتایج

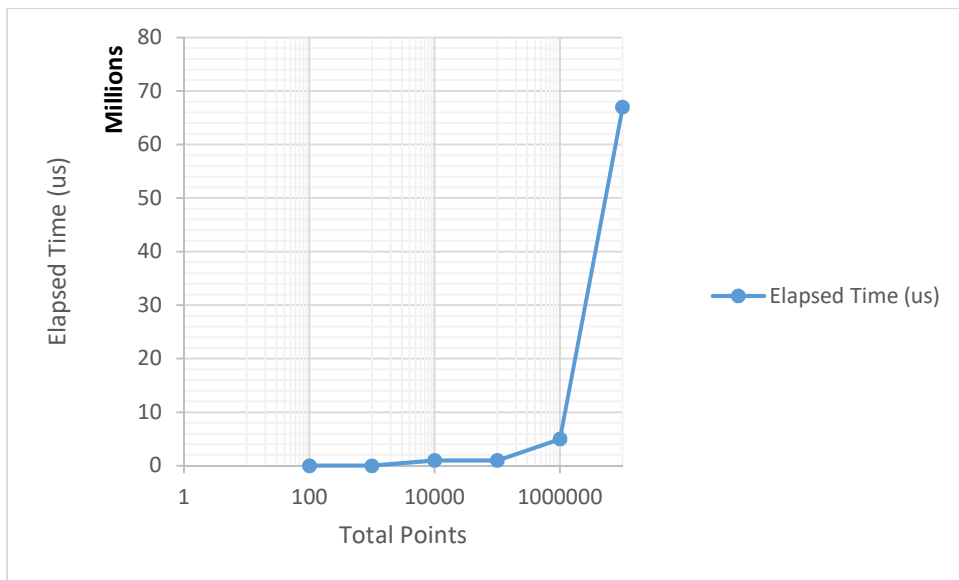
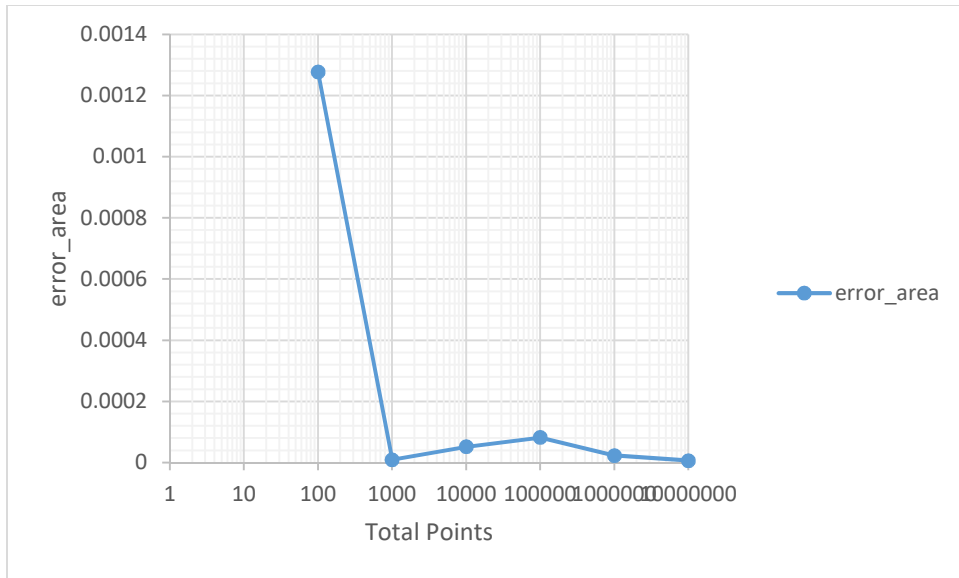
بررسی دقت محاسبه

در این آزمایش، مساحت زیر منحنی در بازه $[0,5.1]$ با استفاده از روش monte-carlo، 330 بخش (chunk) و 8 process تخمین زده شد. تعداد نقاط تصادفی مورد استفاده در هر آزمایش، از 100 نقطه آغاز شده و در هر مرحله 10 برابر افزایش یافته است. نتایج نشان می‌دهند که افزایش تعداد نمونه‌ها باعث بهبود دقت تخمین و کاهش خطا می‌شود. با این حال، برای تعداد نمونه‌های بزرگ، زمان محاسباتی به طور قابل توجهی افزایش می‌یابد. این موضوع بیانگر وجود یک trade-off میان دقت تخمین و زمان اجرای برنامه است؛ به این معنا که افزایش تعداد نمونه‌ها اگرچه دقت نتایج را بهبود می‌بخشد، اما هزینه محاسباتی و زمان اجرا را نیز افزایش می‌دهد.

Results Processes : 8 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 100 Total Area : 0.623300 Elapsed Time : 0.000 Mus	Results Processes : 8 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 1000 Total Area : 0.622032 Elapsed Time : 0.000 Mus	Results Processes : 8 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 10000 Total Area : 0.621970 Elapsed Time : 1000000.000 Mus
Results Processes : 8 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 100000 Total Area : 0.621940 Elapsed Time : 1000000.000 Mus	Results Processes : 8 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 1000000 Total Area : 0.621999 Elapsed Time : 5000000.000 Mus	Results Processes : 8 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 10000000 Total Area : 0.622029 Elapsed Time : 67000000.000 Mus

Processes	Slices	Total Points	Total Area	Elapsed Time (us)	1- COS(b)	error_area
						0.00127774
8	330	100	0.6233	0	0.622022	3
8	330	1000	0.622032	0	0.622022	9.743E-06
						0.00005225
8	330	10000	0.62197	1000000	0.622022	7
8	330	100000	0.62194	1000000	0.622022	8.2257E-05
8	330	1000000	0.621999	5000000	0.622022	2.3257E-05
8	330	10000000	0.622029	67000000	0.622022	6.743E-06

همانطور که دیده میشود، افزایش تعداد نمونه‌ها ابتدا باعث کاهش خطا میشود. اما زمان سپری شده محاسبات از یه بازه ای به صورت نمایی رشد میکند:



بررسی efficiency و speedup

در این بخش، عملکرد موازی برنامه با استفاده از تعداد ثابتی از نقاط مونت کارلو (10,000,000 نقطه) و تعداد متفاوتی از پردازنده‌ها مورد بررسی قرار گرفت. تعداد پردازنده‌ها از 1 تا 128 (به توان‌های 2) افزایش داده شد و زمان اجرای برنامه برای هر حالت اندازه‌گیری گردید.

Results Processes : 1 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 10000000 Total Area : 0.622008 Elapsed Time : 16000000.000 Mus	Results Processes : 2 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 10000000 Total Area : 0.622018 Elapsed Time : 8600000.000 Mus	Results Processes : 4 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 10000000 Total Area : 0.622026 Elapsed Time : 10400000.000 Mus
Results Processes : 8 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 10000000 Total Area : 0.622005 Elapsed Time : 6400000.000 Mus	Results Processes : 16 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 10000000 Total Area : 0.622023 Elapsed Time : 4800000.000 Mus	Results Processes : 32 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 10000000 Total Area : 0.622021 Elapsed Time : 5400000.000 Mus
Results Processes : 64 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 10000000 Total Area : 0.622020 Elapsed Time : 5400000.000 Mus	Results Processes : 128 a : 0.0000 b : 5.1000 Number of slices : 330 Total points : 10000000 Total Area : 0.622018 Elapsed Time : 6400000.000 Mus	

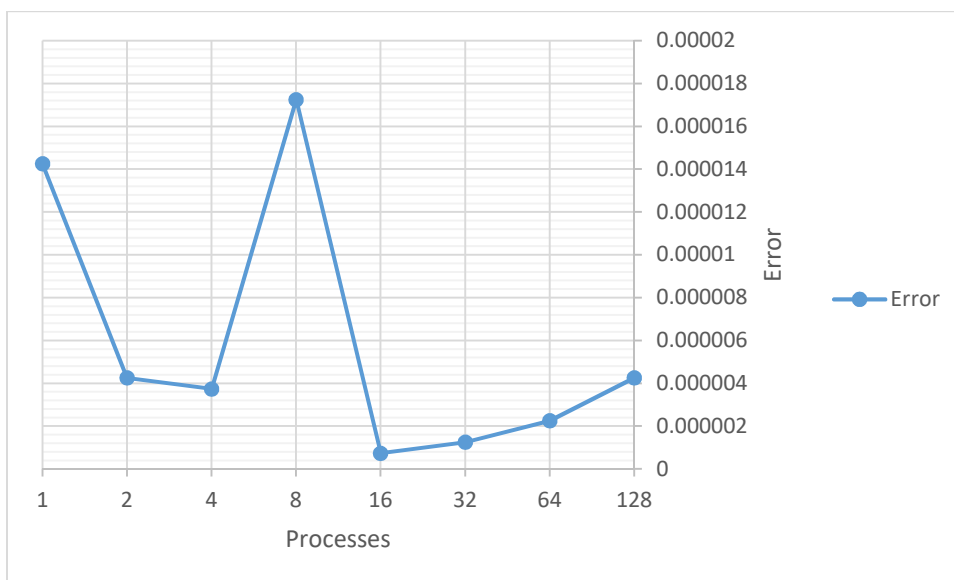
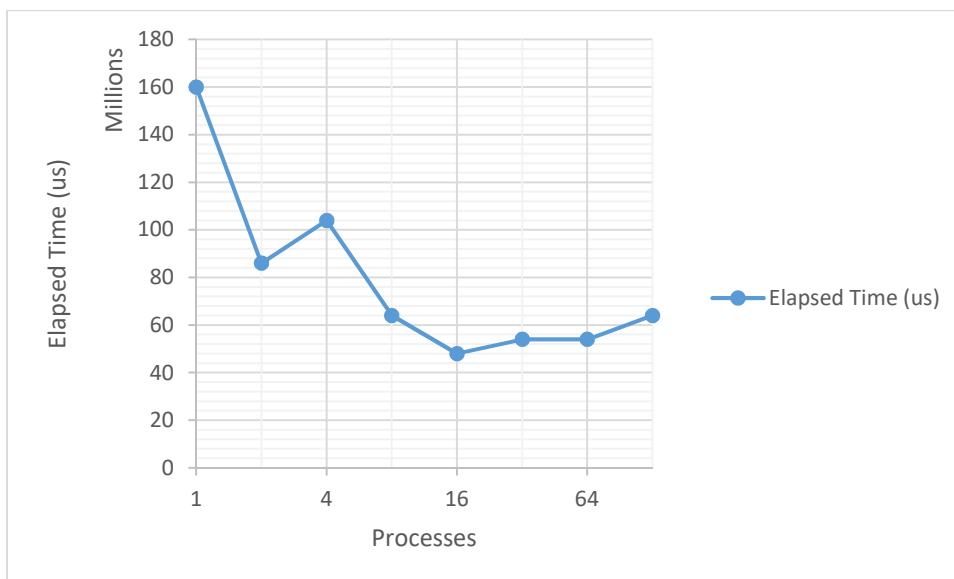
Processes	Slice s	Total Points	Total Area	Elapsed Time (us)	1- COS(b)	Error
						1.4257E-05
1	330	10000000	0.622008	1.6E+08	0.622022	4.257E-06
2	330	10000000	0.622018	86000000	0.622022	3.743E-06
4	330	10000000	0.622026	1.04E+08	0.622022	1.7257E-05
8	330	10000000	0.622005	64000000	0.622022	7.43E-07
16	330	10000000	0.622023	48000000	0.622022	1.257E-06
32	330	10000000	0.622021	54000000	0.622022	2.257E-06
64	330	10000000	0.622020	54000000	0.622022	4.257E-06
128	330	10000000	0.622018	64000000	0.622022	

نتایج نشان می‌دهند که با افزایش تعداد پردازنده‌ها، زمان اجرای برنامه در ابتدا به طور قابل توجهی کاهش می‌یابد. این کاهش زمان ناشی از توزیع بار محاسباتی میان پردازنده‌های بیشتر و استفاده بهتر از منابع محاسباتی موجود است. با این حال، این روند به صورت خطی ادامه پیدا نمی‌کند و پس از تعداد مشخصی از پردازنده‌ها، میزان بهبود عملکرد کاهش می‌یابد.

علت این رفتار آن است که علاوه بر محاسبات، اجرای برنامه شامل هزینه‌های جانبی ناشی از ارتباطات بین پردازنده‌ها، همگام‌سازی (Synchronization) و عملیات جمعی MPI نیز می‌شود. با افزایش تعداد پردازنده‌ها، سهم این سربارها در زمان کل اجرا بیشتر شده و به تدریج به یک عامل محدودکننده تبدیل می‌شود. در نتیجه، حتی در صورت افزایش تعداد پردازنده‌ها، زمان اجرا هرگز به صفر نزدیک نمی‌شود و سرعت‌بخشی (Speedup) به صورت ایده‌آل و متناسب با تعداد پردازنده‌ها رشد نمی‌کند.

همچنین مشاهده می‌شود که افزایش تعداد پردازنده‌ها تأثیری بر دقت روش مونت کارلو ندارد. دلیل این موضوع آن است که دقت تخمین در این روش عمدتاً به تعداد نمونه‌های تصادفی وابسته است و نه به تعداد پردازنده‌های محاسباتی. بنابراین موازی‌سازی تنها زمان مورد نیاز برای انجام محاسبات را کاهش می‌دهد و کیفیت یا دقت نتیجه نهایی را تغییر نمی‌دهد.

به طور کلی، نتایج نشان می‌دهند که برنامه از مقیاس‌پذیری مناسبی برخوردار است، اما با افزایش تعداد پردازنده‌ها، اثر سربارهای ارتباطی و همگام‌سازی باعث کاهش کارایی (Efficiency) شده و مانع دستیابی به سرعت‌بخشی ایده‌آل می‌شود.

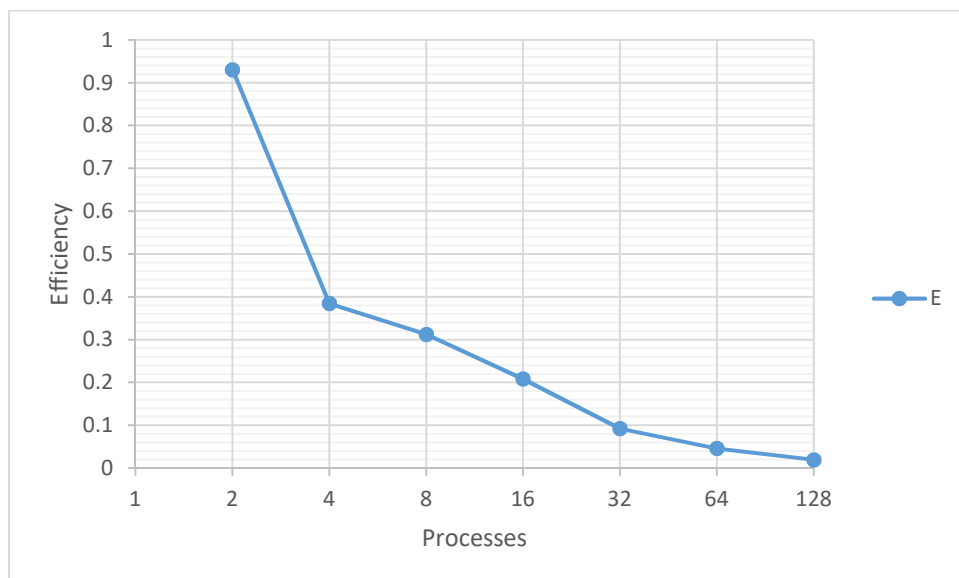
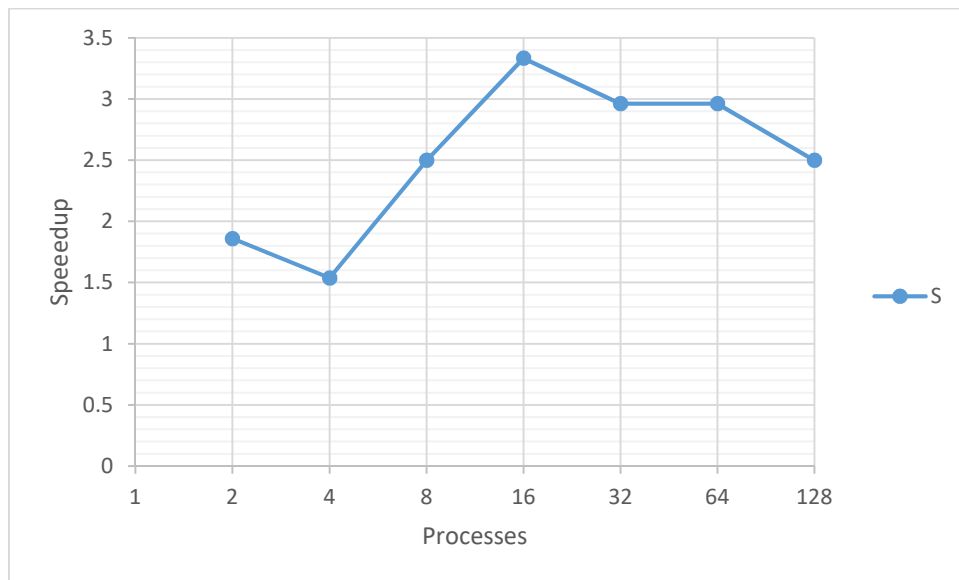


برای محاسبه ی speedup از رابطه ی زیر استفاده میشود. به طوری که زمان سپری شده سری به زمان سپری شده موازی:

$$S(p) = \frac{T_1}{T_p}$$

و برای Efficiency طبق رابطه زیر:

$$E(P) = \frac{S(P)}{P}$$



در پیاده‌سازی حاضر، ارتباط بین Process ها عمدتاً به عملیات جمع‌آوری نتایج نهایی و توزیع داده‌های اولیه محدود می‌شود. در مقایسه با حجم محاسبات انجام‌شده در هر Process، هزینه‌ی ارتباطات در مقادیر بزرگ تعداد نمونه‌های monte-carlo نسبتاً ناچیز است. به همین دلیل، با افزایش تعداد نقاط تصادفی، زمان اجرای برنامه بیشتر تحت تأثیر بار محاسباتی قرار می‌گیرد تا سربار ارتباطات.

با این حال، ارتباطات بین Process ها دارای هزینه‌ی ثابتی هستند که شامل ارسال و دریافت پیام‌ها، همگام‌سازی Process ها و اجرای توابع جمعی MPI می‌شود. هنگامی که حجم محاسبات کم باشد (برای مثال زمانی که تعداد نقاط تصادفی کم است)، این سربار ارتباطی بخش قابل توجهی از زمان کل اجرا را تشکیل می‌دهد و مانع دستیابی به افزایش کارایی متناسب با تعداد Process ها می‌شود.

از دیدگاه مقیاس‌پذیری، برنامه برای مسائل بزرگ تر عملکرد بهتری از خود نشان می‌دهد؛ زیرا با افزایش تعداد نمونه‌ها، نسبت زمان محاسبات به زمان ارتباطات افزایش می‌یابد. در نتیجه، هر Process زمان بیشتری را صرف محاسبات مفید و زمان کمتری را صرف انتظار برای ارتباطات می‌کند. بنابراین انتظار می‌رود با افزایش اندازه مسئله، کارایی موازی و سرعت بخشی (Speedup) برنامه بهبود یابد.

با این وجود، مقیاس‌پذیری برنامه نامحدود نیست. با افزایش تعداد Process ها، حجم محاسبات اختصاص‌یافته به هر Process کاهش می‌یابد، در حالی که هزینه ارتباطات و همگام‌سازی همچنان وجود دارد. در نتیجه پس از یک نقطه مشخص، افزایش تعداد Process ها منجر به کاهش محسوس زمان اجرا نخواهد شد و حتی ممکن است به دلیل غلبه سربار ارتباطات، کارایی برنامه کاهش یابد. این رفتار مطابق با محدودیت‌های مطرح‌شده در Amdahl's Law است که نشان می‌دهد بخش‌های غیرقابل موازی‌سازی و هزینه‌های ارتباطی، حداکثر سرعت بخشی قابل دستیابی را محدود می‌کنند.